

Universidad Autónoma de Tamaulipas

Facultad de Ingeniería Tampico



ASIGNATURA

Programación de Interfaces y Puertos

6to. Semestre – Grupo “I”

2025 -1

TRABAJO

Programas en Clase y Ejercicios

UNIDAD

4 - Desarrollo e Implementación de Controladores Mediante Interfaces y Puertos

Docente: Dr. García Ruiz Alejandro H.

Integrante del Equipo	Nivel de Participación
Izaguirre Cortes Emanuel	33.33
Turrubiates Mejia Gilberto	33.33
García Salas Yahir Misael	33.33
Total:	100%

Índice

<i>Índice</i>	1
<i>Repositorio(s) de Prácticas</i>	2
<i>P1. --- Split Validation ---</i>	2
Descripción de la practica	2
Introducción.....	2
Componentes para el desarrollo de la practica	2
Desarrollo	3
Conclusiones	7
<i>P2. --- Cross-Validation ---</i>	8
<i>Fuentes consultadas (Si aplica)</i>	11

Repositorio(s) de Prácticas

Practica	Repositorio
1 a 2	https://github.com/Emanuel-Izaguirre-Cortes-03/Pip_2025_1_eq_4_i/tree/main/Practicas/Practicas_U4

P1. --- Split Validation ---

Descripción de la practica

Evaluación del Rendimiento de un Modelo de Clasificación para Tipos de Proyecto utilizando Split Validation y Cross-Validation

Introducción

En esta práctica, se abordó el problema de clasificar proyectos en diferentes tipos ('Desarrollo', 'Diseño', 'Investigación') basándose en ciertas características cuantitativas asociadas a ellos: 'Horas Trabajadas', 'Tareas Completadas', 'Miembros Equipo' y 'Días Restantes'. El objetivo principal fue evaluar el rendimiento de un modelo de aprendizaje automático, específicamente un modelo de Regresión Logística, en esta tarea de clasificación. Para obtener una evaluación robusta y confiable, se emplearon dos técnicas fundamentales de validación de modelos: Split Validation (o validación por división) y Cross-Validation (o validación cruzada K-Fold). Estas técnicas permiten estimar cómo el modelo generalizará su capacidad de predicción a datos nuevos e invisibles durante el entrenamiento.

Componentes para el desarrollo de la practica

Para llevar a cabo esta práctica, se utilizaron los siguientes componentes y librerías de Python:

- **pandas:** Librería esencial para la manipulación y análisis de datos, utilizada para leer el archivo Excel que contenía la información de los proyectos y crear un DataFrame.
- **scikit-learn (sklearn):** Una biblioteca de aprendizaje automático integral que proporcionó las herramientas necesarias para:
 - **train_test_split:** Función para dividir el conjunto de datos en conjuntos de entrenamiento y validación para la técnica de Split Validation.

- **LogisticRegression:** El modelo de clasificación seleccionado para predecir el tipo de proyecto.
- **LabelEncoder:** Utilizado para codificar la variable objetivo categórica ('Tipo de Proyecto') en valores numéricos, requeridos por el modelo de Regresión Logística.
- **KFold:** Clase para implementar la estrategia de división K-Fold en la técnica de Cross-Validation.
- **cross_val_score:** Función para realizar la validación cruzada y obtener las puntuaciones de rendimiento (precisión en este caso) en cada fold.
- **cross_val_predict:** Función para obtener las predicciones realizadas por el modelo en cada fold durante la validación cruzada.
- **accuracy_score:** Métrica para evaluar la precisión general del modelo.
- **classification_report:** Función para generar un informe detallado del rendimiento del modelo, incluyendo precisión, recall, F1-score y soporte para cada clase.
- **warnings:** Módulo de Python utilizado para gestionar y suprimir advertencias, en este caso, para limpiar la salida de las advertencias FutureWarning de scikit-learn.
- **Archivo Excel (datos_proyectos.xlsx):** El archivo que contenía los datos de los proyectos, con las características mencionadas y el tipo de proyecto asociado.

Desarrollo

El desarrollo de la práctica se llevó a cabo siguiendo los siguientes pasos:

1. **Carga de datos:** Se leyó el archivo Excel (datos_proyectos.xlsx) utilizando la función `pd.read_excel()` de pandas, creando un DataFrame que contenía la información de los proyectos.
2. **Preparación de datos:**
 - Se separaron las características predictoras (variables independientes: 'Horas Trabajadas', 'Tareas Completadas', 'Miembros Equipo', 'Días Restantes') y la variable objetivo (variable dependiente: 'Tipo de Proyecto').

- La variable objetivo categórica ('Tipo de Proyecto') se codificó a valores numéricos utilizando LabelEncoder, ya que los modelos de aprendizaje automático suelen trabajar con datos numéricos.

3. Split Validation:

- El conjunto de datos se dividió en un conjunto de entrenamiento (80%) y un conjunto de validación (20%) utilizando train_test_split. Se utilizó la estratificación (stratify=y_encoded) para asegurar que la proporción de cada tipo de proyecto fuera similar en ambos conjuntos.
- Se inicializó y entrenó un modelo de Regresión Logística utilizando el conjunto de entrenamiento.
- Se realizaron predicciones sobre el conjunto de validación y se evaluó el rendimiento del modelo utilizando accuracy_score y classification_report.

4. Cross-Validation (K-Fold):

- Se definió una estrategia de validación cruzada K-Fold con k=5 folds utilizando KFold. Se habilitó la mezcla de los datos (shuffle=True) para evitar posibles sesgos.
- Se utilizó cross_val_score para entrenar y evaluar el modelo de Regresión Logística en cada uno de los 5 folds, obteniendo una puntuación de precisión para cada fold. Se calculó la precisión promedio y la desviación estándar de estas puntuaciones.
- Se utilizaron cross_val_predict y classification_report para obtener predicciones y un reporte de clasificación basado en las predicciones realizadas en cada fold cuando actuaba como conjunto de validación.

5. **Supresión de advertencias:** Se importó el módulo warnings y se utilizó warnings.filterwarnings("ignore", category=FutureWarning) para evitar la impresión de advertencias innecesarias en la salida.

6. **Impresión de resultados:** Se imprimieron los nombres de las columnas del DataFrame, las primeras filas de las características, la codificación de la variable objetivo, las clases originales, y los resultados detallados de ambas técnicas de validación (precisión y reporte de clasificación).

Codigo

```
import pandas as pd
from sklearn.model_selection import train_test_split, KFold,
cross_val_score, cross_val_predict
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report
import warnings

# Ignorar las advertencias FutureWarning
warnings.filterwarnings("ignore", category=FutureWarning)
# Especifica la ruta a tu archivo Excel
ruta_excel = 'datos_proyectos.xlsx' # Reemplaza con el nombre de
tu archivo

# Lee el archivo Excel en un DataFrame de pandas
try:
    df = pd.read_excel(ruta_excel)
except FileNotFoundError:
    print(f"Error: No se encontró el archivo en la ruta:
{ruta_excel}")
    exit()

# Imprime los nombres de las columnas para verificar
print("Nombres de las columnas en el DataFrame:")
print(df.columns)

# Separar las características (X) de la variable objetivo (y)
X = df[['Horas Trabajadas', 'Tareas Completadas', 'Miembros
Equipo', 'Dias Restantes']]
y = df['Tipo de Proyecto']

# Codificar la variable objetivo (si no es numérica)
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

print("Características (X):\n", X.head())
print("\nVariable Objetivo Codificada (y_encoded):\n",
y_encoded[:5])
print("\nClases originales:", label_encoder.classes_)

# --- Split Validation ---
X_train, X_val, y_train, y_val = train_test_split(X, y_encoded,
```

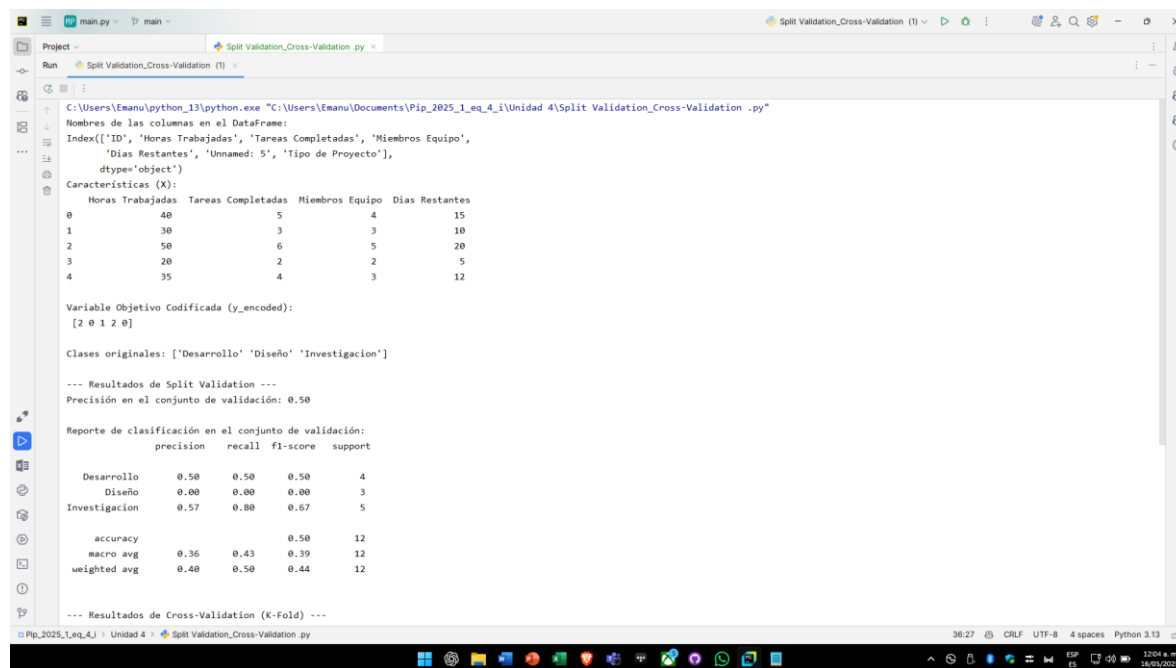
```
test_size=0.2, random_state=42, stratify=y_encoded)

modelo_split = LogisticRegression(random_state=42,
solver='liblinear', multi_class='auto')
modelo_split.fit(X_train, y_train)

y_pred_val = modelo_split.predict(X_val)
accuracy_val = accuracy_score(y_val, y_pred_val)
report_val = classification_report(y_val, y_pred_val,
target_names=label_encoder.classes_)

print("\n--- Resultados de Split Validation ---")
print(f"Precisión en el conjunto de validación:
{accuracy_val:.2f}")
print("\nReporte de clasificación en el conjunto de
validación:\n", report_val)
```

Resultado



```
C:\Users\Emanu\python_13\python.exe "C:\Users\Emanu\Documents\Pip_2025_1_eq_4_i\Unidad 4\Split Validation_Cross-Validation -py"
Names de las columnas en el DataFrame:
Index(['ID', 'Horas Trabajadas', 'Tareas Completadas', 'Miembros Equipo',
'Dias Restantes', 'Unnamed: 5', 'Tipo de Proyecto'],
      dtype='object')
Características (X):
   Horas Trabajadas  Tareas Completadas  Miembros Equipo  Dias Restantes
0                40                   5                4                15
1                30                   3                3                10
2                50                   6                5                20
3                20                   2                2                 5
4                35                   4                3                12

Variable Objetivo Codificada (y_encoded):
[2 0 1 2 0]

Clases originales: ['Desarrollo' 'Diseño' 'Investigacion']

--- Resultados de Split Validation ---
Precisión en el conjunto de validación: 0.50

Reporte de clasificación en el conjunto de validación:
      precision    recall  f1-score   support

Desarrollo      0.50      0.50      0.50         4
Diseño          0.00      0.00      0.00         3
Investigacion    0.57      0.80      0.67         5

 accuracy          0.50         12
 macro avg         0.36         12
 weighted avg       0.40         12

--- Resultados de Cross-Validation (K-Fold) ---
```

Conclusiones

Basándonos en los resultados obtenidos:

- El modelo de Regresión Logística logró una precisión de aproximadamente el 50% en el conjunto de validación (Split Validation) y una precisión promedio de alrededor del 52% utilizando la validación cruzada K-Fold. Esto sugiere que el modelo tiene una capacidad moderada para clasificar los tipos de proyecto basándose en las características proporcionadas.
- El reporte de clasificación reveló un rendimiento desigual entre las diferentes clases de tipos de proyecto. La clase 'Diseño' parece ser la más difícil de predecir, con una precisión y recall bajos en ambos métodos de validación. Las clases 'Desarrollo' e 'Investigacion' mostraron un rendimiento ligeramente mejor.
- La variabilidad en las puntuaciones de precisión entre los folds en la validación cruzada indica que el rendimiento del modelo puede depender de la partición específica de los datos.
- La diferencia relativamente pequeña entre los resultados de Split Validation y Cross-Validation sugiere que la división inicial de los datos en la validación por división no fue particularmente sesgada en este caso. Sin embargo, la validación cruzada proporciona una evaluación más robusta.

Posibles pasos futuros:

- **Explorar otros modelos de clasificación:** Probar algoritmos diferentes como Support Vector Machines, Random Forests, o Gradient Boosting podría llevar a un mejor rendimiento.
- **Ingeniería de características:** Crear nuevas características a partir de las existentes o transformar las características actuales podría mejorar la capacidad del modelo para distinguir entre los tipos de proyecto.
- **Ajuste de hiperparámetros:** Optimizar los parámetros del modelo de Regresión Logística o de cualquier otro modelo que se pruebe podría mejorar su rendimiento.

- **Análisis del desbalance de clases:** Investigar si existe un desbalance significativo en el número de proyectos por tipo en el conjunto de datos y aplicar técnicas para mitigar sus efectos (si es el caso).
- **Recolección de más datos:** Un conjunto de datos más grande y diverso podría ayudar al modelo a aprender patrones más complejos y generalizar mejor.

Esta práctica proporcionó una comprensión fundamental de cómo evaluar el rendimiento de un modelo de clasificación utilizando técnicas de validación comunes y cómo interpretar los resultados obtenidos. Los resultados sugieren que el modelo actual podría beneficiarse de mejoras adicionales para lograr una clasificación más precisa de los tipos de proyecto.

P2. --- Cross-Validation ---

Cross-Validation (Validación Cruzada K-Fold):

Este método dividió tu conjunto de datos en $k=5$ partes (folds). Luego, entrenó y evaluó el modelo 5 veces. En cada iteración, usó 4 folds para entrenar y el fold restante para evaluar. Esto proporciona una estimación más robusta del rendimiento del modelo, ya que se evalúa en diferentes subconjuntos de los datos.

Resultados de Cross-Validation (K-Fold):

- **Puntuaciones de precisión en cada fold: [0.33, 0.50, 0.83, 0.25, 0.67]:** Estas son las precisiones obtenidas en cada una de las 5 iteraciones de la validación cruzada. Observa que hay variabilidad en el rendimiento entre los diferentes folds.
- **Precisión promedio de validación cruzada: 0.52:** Esta es la media de las precisiones obtenidas en cada fold. Proporciona una estimación más general del rendimiento esperado del modelo en datos no vistos. Es ligeramente mejor que la precisión obtenida en una única división de validación.
- **Desviación estándar de las puntuaciones: 0.21:** Esto mide la dispersión de las precisiones entre los diferentes folds. Una desviación estándar más alta indica que el rendimiento del modelo es más sensible a cómo se dividen los datos.
- **Reporte de clasificación con Cross-Validation:** Este reporte se genera al combinar las predicciones de todos los folds (cada instancia se predice cuando está en el fold

de validación). Proporciona una visión general del rendimiento del modelo en todo el conjunto de datos mediante validación cruzada:

- La precisión, recall y F1-score promedio para cada clase son más estables y representan el rendimiento general esperado del modelo.
- Observamos que la clase 'Diseño' sigue siendo la más difícil de predecir (F1-score bajo).

Interpretación de los resultados de Cross-Validation:

La validación cruzada sugiere un rendimiento promedio del modelo alrededor del 52% de precisión. Al igual que en la validación por división, la clase 'Diseño' presenta el mayor desafío para el modelo. La variabilidad en las puntuaciones de los folds (desviación estándar de 0.21) indica que el rendimiento del modelo puede fluctuar dependiendo de la partición específica de los datos.

Comparación de los dos métodos:

- La validación cruzada proporciona una estimación más confiable del rendimiento del modelo que una única división de validación, especialmente cuando el conjunto de datos es relativamente pequeño.
- La validación por división es más rápida computacionalmente, pero su resultado puede ser más sensible a la forma en que se realiza la división.

En conclusión:

El modelo de Regresión Logística actual tiene una precisión alrededor del 50-52% para predecir el tipo de proyecto. Tiene dificultades significativas para identificar proyectos de 'Diseño'. Podrías considerar explorar otros modelos, ajustar los hiperparámetros de la Regresión Logística o investigar si las características actuales son suficientes para distinguir claramente entre los diferentes tipos de proyectos.

Codigo

```
# --- Cross-Validation (K-Fold) ---
modelo_cv = LogisticRegression(random_state=42,
solver='liblinear', multi_class='auto')
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores_cv = cross_val_score(modelo_cv, X, y_encoded, cv=kf,
scoring='accuracy')

print("\n--- Resultados de Cross-Validation (K-Fold) ---")
print("Puntuaciones de precisión en cada fold:", scores_cv)
print(f"Precisión promedio de validación cruzada:
{scores_cv.mean():.2f}")
print(f"Desviación estándar de las puntuaciones:
{scores_cv.std():.2f}")

y_pred_cv = cross_val_predict(modelo_cv, X, y_encoded, cv=kf)
report_cv = classification_report(y_encoded, y_pred_cv,
target_names=label_encoder.classes_)
print("\nReporte de clasificación con Cross-Validation:\n",
report_cv)
```

Este codigo esta en compañía del codigo de arriba

Su resultado de este es

```
--- Resultados de Cross-Validation (K-Fold) ---
Puntuaciones de precisión en cada fold: [0.33333333 0.5          0.83333333 0.25         0.66666667]
Precisión promedio de validación cruzada: 0.52
Desviación estándar de las puntuaciones: 0.21

Reporte de clasificación con Cross-Validation:
      precision    recall  f1-score   support

Desarrollo      0.53      0.44      0.48        18
Diseño          0.38      0.33      0.35        18
Investigacion   0.59      0.71      0.64        24

accuracy              0.52        60
macro avg      0.50      0.50      0.49        60
weighted avg   0.51      0.52      0.51        60
```

Fuentes consultadas **(Si aplica)**

Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (2nd ed.). O'Reilly Media.

Müller, A. C., & Guido, S. (2016). *Introduction to Machine Learning with Python*. O'Reilly Media.

pandas Development Team. (n.d.). *pandas User Guide*. Recuperado de https://pandas.pydata.org/docs/user_guide/index.html

Python Software Foundation. (n.d.). *warnings — Warning control*. Python 3.13.0 documentation. Recuperado de <https://docs.python.org/3/library/warnings.html>

scikit-learn developers. (n.d.). *scikit-learn: Machine Learning in Python*. Recuperado de https://scikit-learn.org/stable/user_guide.html